



31.1. Python: Pandas DataFrame

Where data stops being a column and becomes a dataset.

Previous Section:

[!\[\]\(c3d993ca47bfe2a953c700506ce31fa0_img.jpg\) 31. Python: Pandas Series](#)

Contents:

[1. Introduction to Pandas DataFrame](#)

[1.1. Structure of a DataFrame](#)

[1.2. Creating a DataFrame](#)

[1.3. Naming Columns](#)

[1.4. Naming the Index](#)

[2. Common Operations on Pandas DataFrame](#)

[2.1. Displaying DataFrame Information](#)

[2.2. Indexing and Slicing](#)

[2.3. Filtering a DataFrame](#)

[2.4. Sorting a DataFrame](#)

[2.5. Modifying a DataFrame](#)

[2.6. Data Types in DataFrame](#)

[2.7. Statistical Methods](#)

[2.8. Data Preprocessing](#)

[2.9. Some Advanced Techniques](#)

[Summary](#)

[References](#)

In the previous section, we explored the **Series**, Pandas' one-dimensional data structure.

- A Series is useful when working with a single collection of values. But most real-world data is not stored in a single column.
- A student record contains names, majors, GPAs, and ages.
- A sales report contains products, quantities, prices, and profits.
- A Machine Learning dataset may contain dozens—or even hundreds—of features describing each sample.

Working with each column separately would quickly become impractical. This is why Pandas provides its most important data structure: the **DataFrame**.

A DataFrame allows us to organize multiple Series together into a table-like structure containing rows and columns. It resembles a spreadsheet, a database table, or a two-dimensional array—but with significantly more power and flexibility. In fact, when people say they are "using Pandas", they are usually working with DataFrames.

Throughout Data Analytics, Data Science, and Machine Learning, DataFrames will become your primary tool for storing, exploring, cleaning, transforming, and analyzing datasets.

1. Introduction to Pandas DataFrame

A **DataFrame** is a two-dimensional data structure that stores information in the form of rows and columns. You can think of it as a spreadsheet inside Python.

For example:

Name	Major	GPA
Alice	CS	3.8
Bob	IT	3.5
Charlie	AI	3.9

Each row represents a record (sample).

Each column represents a feature (attribute).

Unlike ordinary tables, Pandas DataFrames provide powerful tools for searching, filtering, modifying, and analyzing data.

1.1. Structure of a DataFrame

A DataFrame contains three major components:

- **Columns:** The features or attributes being stored. E.g., Name; Major; GPA
- **Rows:** Individual records or samples. E.g., Alice; Bob; Charlie
- **Data:** The actual values stored inside the table.

```
Alice | CS | 3.8  
Bob   | IT | 3.5
```

1.2. Creating a DataFrame

The main constructor is: `pd.DataFrame()` . Pandas can create DataFrames from many different data sources.

- From a Dictionary of Lists: This is the most common approach. Each key becomes a column. Each list becomes the column values.

```
import pandas as pd  
  
data = {"Name": ["Alice", "Bob", "Charlie"],  
        "Major": ["CS", "IT", "AI"],  
        "GPA": [3.8, 3.5, 3.9]}  
  
df = pd.DataFrame(data)  
print(df)
```

```
   Name Major  GPA  
0  Alice   CS  3.8  
1   Bob   IT  3.5  
2 Charlie   AI  3.9
```

- From a List of Dictionaries: Each dictionary becomes one row.

```
import pandas as pd

data = [{"Name": "Alice", "Major": "CS", "GPA": 3.8},
        {"Name": "Bob", "Major": "IT", "GPA": 3.5},
        {"Name": "Charlie", "Major": "AI", "GPA": 3.9}]

df = pd.DataFrame(data)
print(df)
```

```
   Name Major  GPA
0  Alice   CS  3.8
1   Bob   IT  3.5
2 Charlie   AI  3.9
```

- From a List of Lists

```
import pandas as pd

data = [
    ["Alice", "CS", 3.8],
    ["Bob", "IT", 3.5],
    ["Charlie", "AI", 3.9]
]

df = pd.DataFrame(data, columns=["Name", "Major", "GPA"])
print(df)
```

```
   Name Major  GPA
0  Alice   CS  3.8
1   Bob   IT  3.5
2 Charlie   AI  3.9
```

- From NumPy Arrays

```
import pandas as pd
import numpy as np

data = np.array([[85, 90],
                 [88, 92],
                 [95, 97]])

df = pd.DataFrame(data, columns=["Math", "Science"])
print(df)
```

```
Math Science
0  85     90
1  88     92
2  95     97
```

- From External Data Sources - Pandas can also load data directly from:
 - CSV files
 - Excel spreadsheets
 - SQL Databases
 - APIs
 - Other data formats

We will explore these methods in the Data Analytics course.

1.3. Naming Columns

- The `columns=` parameter allows us to specify column names.

```
import pandas as pd

data = [ ["Alice", "CS"], ["Bob", "IT"] ]

df = pd.DataFrame(data, columns=["Name", "Major"])
print(df)
```

- Interesting Behavior of `columns=` : Suppose some column names exist while others do not.

```
import pandas as pd

data = {"Name": ["Alice", "Bob"],
        "Major": ["CS", "IT"]}

df = pd.DataFrame(data, columns=["Name", "Major", "GPA"])
print(df)
```

```
   Name Major GPA
0  Alice   CS  NaN
1   Bob   IT  NaN
```

If a requested column does not exist, Pandas creates the column, and all values become `NaN`. If `columns=` is omitted, Pandas automatically uses the keys of the input data.

1.4. Naming the Index

By default, indexes are: `0, 1, 2, 3, ...` . We can customize them using `index=` .

```
import pandas as pd

data = {"Name": ["Alice", "Bob", "Charlie"]}

df = pd.DataFrame(data, index=["a", "b", "c"])
print(df)
```

```
   Name
a  Alice
b   Bob
c Charlie
```

2. Common Operations on Pandas DataFrame

The Data Analytics course will dive deeply into DataFrame operations. For now, we will focus on the most common and useful methods.

2.1. Displaying DataFrame Information

- Index Labels

```
print(df.index)
```

- Column Names

```
print(df.columns)
```

- Convert to Python Lists

```
print(df.columns.tolist())
```

- Convert to Dictionary

```
print(df.to_dict())
```

- Set a Column as the Index

```
df = df.set_index("Name")  
print(df)
```

	Major
Name	
Alice	CS
Bob	IT
Charlie	AI

- Restore Default Index

```
df = df.reset_index()  
print(df)
```

- Number of Rows

```
print(len(df))
```

- Number of Columns

```
print(len(df.columns))
```

- Shape

```
print(df.shape)
```

(3, 2) → (rows, columns)

- Total Number of Elements

```
print(df.size)
```

rows × columns

- Retrieve a Column

```
data = {"Name": ["Alice", "Bob"],  
        "Major": ["CS", "IT"]}
```

```
df = pd.DataFrame(data)
```

```
print(df["Major"])
```

Notice that the result is a **Series**.

- Rename Columns

```
df = df.rename(columns={"Major": "AI"})  
print(df)
```


- DataFrame Information

```
df.info()
```

This displays: Number of rows; Number of columns; Data types; Non-missing values

- Statistical Summary

```
print(df.describe())
```

This returns: Count; Mean; Standard Deviation; Minimum; Quartiles; Maximum

- Include Non-Numerical Columns

```
print(df.describe(include="all"))
```

- First Rows

```
print(df.head())  
# Default df.head(5)
```

- Last Rows

```
print(df.tail())  
# Default df.tail(5)
```

2.2. Indexing and Slicing

Pandas provides two major indexing systems. Consider the following dataframe:

```
data = {"Name": ["Alice", "Bob", "James", "Matt", "Cindy"],
        "Age": [18, 19, 19, 18, 18],
        "Major": ["CS", "IT", "AI", "IT", "AI"]}
df = pd.DataFrame(data)
```

- Integer-Based Indexing: `iloc` . Uses row and column positions →
`df.iloc[row_position, column_position]`

```
# Second row, third column
print(df.iloc[1, 2])
```

IT

- Slicing with `iloc`

```
print(df.iloc[0:2, 0:2])
```

```
Name Age
0 Alice 18
1 Bob 19
```

Returns: Rows 0 and 1; Columns 0 and 1. The ending positions are excluded.

- Label-Based Indexing: `loc` . Uses row and column names. To use row labels effectively, we usually set a custom index first.

```
df = df.set_index("Name")
print(df.loc["Bob", "Major"])
```

IT

- Slicing with `loc`

```
print(df.loc[["Alice", "James"], ["Major"]])
```

```
Name  Major  
Alice  CS  
James  AI
```

Unlike `iloc`, the ending labels are included. This is one of the biggest differences between the two methods.

At this point, we have officially moved beyond single-column data structures. A **Series** stores one labeled column of data. A **DataFrame** combines multiple Series into a complete dataset.

2.3. Filtering a DataFrame

One of the most powerful features of Pandas is the ability to retrieve only the rows that satisfy specific conditions. Instead of manually looping through every row, we can simply describe the condition we want and let Pandas perform the filtering for us.

A filtering operation generally works in two steps:

1. Create a filtering condition.
2. Use that condition to retrieve matching rows → `df[filtering_condition]`

The filtering condition produces a **Boolean Series**: `True` → Row matches the condition. `False` → Row does not match the condition. Pandas then returns only the rows marked as `True`.

Consider the following DataFrame:

```
import pandas as pd

data = {"Name": ["Alice", "Bob", "Charlie", "David"],
        "Major": ["CS", "IT", "CS", "AI"],
        "GPA": [3.8, 3.2, 3.9, 3.5]}

df = pd.DataFrame(data)

print(df)
```

```
   Name Major  GPA
0  Alice   CS  3.8
1   Bob   IT  3.2
2 Charlie   CS  3.9
3  David   AI  3.5
```

- Comparative Conditions - These compare values using operators such as:

```
>; <; >=; <=; ==; !=
```

```
print(df[df["GPA"] >= 3.7])
```

```
   Name Major  GPA
0  Alice   CS  3.8
2 Charlie   CS  3.9
```

- Logical Conditions - Multiple conditions can be combined using: `& (And); | (Or); ~ (Not)`

```
print(df[(df["Major"] == "CS") & (df["GPA"] >= 3.8)])
```

```
   Name Major  GPA
0  Alice   CS  3.8
2 Charlie   CS  3.9
```

```
print(df[(df["Major"] == "AI") | (df["Major"] == "IT")])
```

Name Major GPA

```
1 Bob IT 3.2
3 David AI 3.5
```

```
print(df[~(df["Major"] == "IT")])
```

Name Major GPA

```
0 Alice CS 3.8
2 Charlie CS 3.9
3 David AI 3.5
```

- Subset-Based Conditions with `isin()` : Checks whether a value belongs to a collection.

```
print(df[df["Major"].isin(["CS", "AI"])])
```

Name Major GPA

```
0 Alice CS 3.8
2 Charlie CS 3.9
3 David AI 3.5
```

- `any()` and `all()` : These are commonly used when working with multiple Boolean conditions.

```
print((df["GPA"] > 3.5).any())
```

True

```
print((df["GPA"] > 3.5).all())
```

False

- String Conditions

When working with text columns, Pandas provides string-based filtering methods.

```
# str.contains()
print(df[df["Name"].str.contains("a")])

# str.startswith()
print(df[df["Name"].str.startswith("A")])

# str.endswith()
print(df[df["Name"].str.endswith("d")])
```

```
Name Major GPA
2 Charlie  CS 3.9
3  David   AI 3.5
```

```
Name Major GPA
0 Alice   CS 3.8
```

```
Name Major GPA
3 David   AI 3.5
```

2.4. Sorting a DataFrame

Sorting allows us to organize data based on values or indexes.

- Sort by Values: The `sort_values()` method sorts rows based on one or more columns.

```
print(df.sort_values("GPA"))
```

```
Name Major GPA
1  Bob   IT 3.2
3  David  AI 3.5
0  Alice  CS 3.8
2  Charlie CS 3.9
```

- Descending order:

```
print(df.sort_values("GPA", ascending=False))
```

```
Name Major GPA
2 Charlie  CS 3.9
0 Alice   CS 3.8
3 David   AI 3.5
1 Bob     IT 3.2
```

- Sorting by Multiple Columns

```
print(df.sort_values(["Major", "GPA"]))
```

```
Name Major GPA
3 David   AI 3.5
0 Alice   CS 3.8
2 Charlie CS 3.9
1 Bob     IT 3.2
```

- Sort by Index

```
# By default, indexes are sorted in ascending order.
print(df.sort_index())
```

```
Name Major GPA
0 Alice   CS 3.8
1 Bob     IT 3.2
2 Charlie CS 3.9
3 David   AI 3.5
```

2.5. Modifying a DataFrame

DataFrames can be modified by adding, removing, or updating rows and columns.

- Adding a Column: If a column does not exist, assignment automatically creates it.

```
df["Status"] = "Active"  
print(df)
```

	Name	Major	GPA	Status
0	Alice	CS	3.8	Active
1	Bob	IT	3.2	Active
2	Charlie	CS	3.9	Active
3	David	AI	3.5	Active

- Adding a Row using `loc[len(df)]`.

```
df.loc[len(df)] = ["Eve", "DS", 3.7, "Active"]  
print(df)
```

	Name	Major	GPA	Status
0	Alice	CS	3.8	Active
1	Bob	IT	3.2	Active
2	Charlie	CS	3.9	Active
3	David	AI	3.5	Active
4	Eve	DS	3.7	Active

- Removing Columns

```
df = df.drop(columns=["Status"])  
print(df)
```

	Name	Major	GPA
0	Alice	CS	3.8
1	Bob	IT	3.2
2	Charlie	CS	3.9
3	David	AI	3.5
4	Eve	DS	3.7

- Removing Rows


```
df = df.drop(index=1)
print(df)
```

```
Name Major GPA
0  Alice  CS  3.8
2  Charlie CS  3.9
3  David  AI  3.5
4   Eve   DS  3.7
```

- `pop()` : Removes a column and returns it as a Series.

```
major_column = df.pop("Major")
print(major_column)
```

```
0  CS
2  CS
3  AI
4  DS
Name: Major, dtype: object
```

- Updating an Entire Column

```
# All rows receive the same value.
df["GPA"] = 4.0
print(df)
```

```
Name GPA
0  Alice 4.0
2  Charlie 4.0
3  David 4.0
4   Eve 4.0
```

- Updating Selected Rows: Suppose we only want to update CS students.

```
data = {"Name": ["Alice", "Bob", "Charlie", "David"],
        "Major": ["CS", "IT", "CS", "AI"],
        "GPA": [3.8, 3.2, 3.9, 3.5]}

df = pd.DataFrame(data)

# Only matching rows are modified.
df.loc[df["Major"] == "CS", "GPA"] = 4.0
print(df)
```

```
Name Major GPA
0  Alice  CS  4.0
1   Bob  IT  3.2
2 Charlie  CS  4.0
3  David  AI  3.5
```

- Creating a New DataFrame with Filtering

```
honor_students = df[df["GPA"] >= 3.8]
print(honor_students)
```

```
Name Major GPA
0  Alice  CS  4.0
2 Charlie  CS  4.0
```

2.6. Data Types in DataFrame

Like Series, every DataFrame column has a data type.

Common Pandas data types include:

Type	Description
int64	Integers
float64	Floating-point numbers
object	Strings and mixed values
bool	Boolean values

- Determine the Object Type

```
print(type(df))
```

```
<class 'pandas.core.frame.DataFrame'>
```

- Display Column Data Types

```
print(df.dtypes)
```

```
Name    object
Major    object
GPA    float64
dtype: object
```

- Selecting Columns by Data Type

Numeric columns only:

```
print(df.select_dtypes(include=["number"]))
```

```
GPA
0 4.0
1 3.2
2 4.0
3 3.5
```

Text columns only:

```
print(df.select_dtypes(include=["object"]))
```

```
Name Major
0  Alice  CS
1   Bob   IT
2 Charlie  CS
3  David  AI
```

2.7. Statistical Methods

Pandas provides a rich collection of mathematical and statistical operations for both Series and DataFrames.

- Arithmetic Operations - Element-wise addition:

```
import pandas as pd

data = {"A": [1, 2, 3], "B": [4, 5, 6]}

df = pd.DataFrame(data)
print(df + 10)
```

```
   A  B
0  11 14
1  12 15
2  13 16
```

Equivalent methods:

```
print("Add:\n", df["A"].add(df["B"]))
print("\nSubtract:\n", df["A"].sub(df["B"]))
print("\nMultiply:\n", df["A"].mul(df["B"]))
print("\nDivide:\n", df["A"].div(df["B"]))
```

Add:

0 5

1 7

2 9

dtype: int64

Subtract:

0 -3

1 -3

2 -3

dtype: int64

Multiply:

0 4

1 10

2 18

dtype: int64

Divide:

0 0.25

1 0.40

2 0.50

dtype: float64

- Descriptive Statistics

```
print("Sum:\n", df.sum())
print("Mean:\n", df.mean())
print("Median:\n", df.median())
print("Mode:\n", df.mode())
print("Quantile:\n", df.quantile(0.5))
print("Max:\n", df.max())
print("Min:\n", df.min())
```

```
print("Variance:\n", df.var())
print("Standard Deviation:\n", df.std())
```

These methods compute: Sum; Mean; Median; Mode; Quantiles; Maximum Value, Minimum Value, Variance; Standard Deviation, for every numerical column.

- Largest values and smallest values

```
print("Largest values:\n",df.nlargest(2, "A"))
print("Smallest values:\n",df.nsmallest(2, "A"))
```

Largest values:

```
A B
2 3 6
1 2 5
```

Smallest values:

```
A B
0 1 4
1 2 5
```

- Counting Values

```
print("Count Non-Null Values:\n", df.count())
# For a Series:
print("Count Unique Values:\n", df["A"].value_counts())
```

Count Non-Null Values:

```
A 3
B 3
dtype: int64
```

Count Unique Values:

```
A
1 1
2 1
3 1
```

Name: count, dtype: int64

- **Applying Functions:** The `apply()` method allows us to apply custom functions to DataFrame columns.

```
print(df["A"].apply(lambda x: x * 2))
```

```
0    2
```

```
1    4
```

```
2    6
```

```
Name: A, dtype: int64
```

This is one of the most flexible tools in Pandas and becomes especially useful when performing custom transformations on data.

2.8. Data Preprocessing

Real-world data is rarely perfect. Datasets often contain missing values, duplicated entries, inconsistent labels, and categorical values that must be transformed before analysis or Machine Learning.

The process of preparing data for analysis is known as **data preprocessing**. Pandas provides many tools that make preprocessing straightforward and efficient.

- **Missing Values:** A **missing value** is a value that is unknown, unavailable, or not recorded. Pandas represents missing values using `NaN`, which stands for **Not a Number**.

Consider the following DataFrame:

```
import pandas as pd

data = {"Name": ["Alice", "Bob", "Charlie"],
        "GPA": [3.8, None, 3.9]}

df = pd.DataFrame(data)
print(df)
```

```
   Name  GPA
0  Alice  3.8
1   Bob  NaN
2 Charlie  3.9
```

Detect Missing Values: Use `isna()` or `isnull()`.

```
print(df.isna())
```

```
   Name  GPA
0  False  False
1  False   True
2  False  False
```

Both methods are equivalent: `df.isna()` and `df.isnull()`

Filling Missing Values: Use `fillna()`.

```
# df["GPA"] = df["GPA"].fillna(0)
# Fill in the missing value with mean
df["GPA"] = df["GPA"].fillna(df["GPA"].mean())
print(df)
```

```
   Name  GPA
0  Alice  3.80
1   Bob  3.85
2 Charlie  3.90
```

Dropping Missing Values using `dropna()`.


```
print(df.dropna())
```

Rows containing missing values are removed.

- Unique Values help us understand the distinct categories present in a dataset.

Retrieve Unique Values

```
import pandas as pd

data = {"Major": ["CS", "IT", "CS", "AI", "IT"]}

df = pd.DataFrame(data)
print(df["Major"].unique())
```

['CS' 'IT' 'AI']

Count Unique Values

```
print(df["Major"].nunique())
```

3

- Duplicate Values: Duplicate records are common in large datasets. Pandas allows us to detect and remove them easily.

Detect Duplicates

```
# True indicates a duplicated row.  
print(df.duplicated())
```

```
0  False  
1  False  
2   True  
3  False  
4   True  
dtype: bool
```

Remove Duplicates

```
# By default, the first occurrence is kept.  
print(df.drop_duplicates())
```

```
0  CS  
1  IT  
3  AI
```

- Replacing Values: Sometimes we need to substitute values throughout a DataFrame. We do that using `replace()`.

```
import pandas as pd  
  
data = {"Major": ["CS", "IT", "AI"]}  
  
df = pd.DataFrame(data)  
df = df.replace({"CS": "Computer Science"})  
print(df)
```

```
      Major  
0  Computer Science  
1         IT  
2         AI
```

- Mapping Values: Mapping converts one value into another according to a specified dictionary. This is especially useful for **label encoding**. The `map()` method works on a Series.

```
import pandas as pd

data = {"Major": ["CS", "IT", "AI"]}
df = pd.DataFrame(data)

encoding = {"CS": 0, "IT": 1, "AI": 2}
df["Major_Code"] = df["Major"].map(encoding)
print(df)
```

```
Major Major_Code
0  CS         0
1  IT         1
2  AI         2
```

- One-Hot Encoding: Machine Learning algorithms often require numerical data. One-Hot Encoding converts a categorical column into multiple binary columns. We do this using `pd.get_dummies()`.

```
import pandas as pd

data = {"Major": ["CS", "IT", "AI"]}

df = pd.DataFrame(data)

encoded = pd.get_dummies(df)
print(encoded)
```

```
Major_AI Major_CS Major_IT
0  False    True  False
1  False   False   True
2   True   False  False
```

Each category becomes its own column.

2.9. Some Advanced Techniques

The following techniques are extremely important in Data Analytics and Data Science. We will only introduce them briefly here and revisit them in much greater depth in the Data Analytics course.

- **Merging:** Combines DataFrames based on one or more common columns.

Using `df.merge()`

```
students = pd.DataFrame({"ID": [1, 2, 3],
                          "Name": ["Alice", "Bob", "Charlie"]})

scores = pd.DataFrame({"ID": [1, 2, 3],
                       "GPA": [3.8, 3.5, 3.9]})

result = students.merge(scores, on="ID")
print(result)
```

	ID	Name	GPA
0	1	Alice	3.8
1	2	Bob	3.5
2	3	Charlie	3.9

- **Joining:** Combines DataFrames using their indexes. Using `df.join()`

```
df1 = pd.DataFrame({"Name": ["Alice", "Bob"]}, index=[1, 2])
df2 = pd.DataFrame({"GPA": [3.8, 3.5]}, index=[1, 2])
print(df1.join(df2))
```

	Name	GPA
1	Alice	3.8
2	Bob	3.5

- **Aggregation:** Groups data and calculates statistical summaries. Using

`df.groupby()`

```
data = {"Major": ["CS", "CS", "IT"],
        "GPA": [3.8, 3.6, 3.5]}

df = pd.DataFrame(data)
print(df.groupby("Major")["GPA"].mean())
```

```
Major
CS    3.7
IT    3.5
```

- **Concatenation:** Combines DataFrames vertically or horizontally. Using

```
pd.concat()
```

```
df1 = pd.DataFrame({"A": [1, 2]})
df2 = pd.DataFrame({"A": [3, 4]})

result = pd.concat([df1, df2])
print(result)
```

```
A
0 1
1 2
0 3
1 4
```

- **Segmentation:** Segmentation divides continuous numerical values into ranges (bins). Using `pd.cut()`

```
import pandas as pd

scores = [55, 68, 75, 88, 95]

grades = pd.cut(scores, bins=[0, 60, 70, 80, 100],
                labels=["F", "D", "C", "A"])

print(grades)
```

```
['F', 'D', 'C', 'A', 'A']
```

```
Categories (4, object): ['F' < 'D' < 'C' < 'A']
```

Summary

In this chapter, we explored the **Pandas DataFrame**, the most important data structure in the Pandas ecosystem.

We learned how to create DataFrames from dictionaries, lists, NumPy arrays, and other data sources. We examined their structure, including rows, columns, indexes, and values, and learned how to inspect and manipulate datasets using a variety of built-in methods.

Along the way, we covered:

- Creating and configuring DataFrames
- Retrieving rows, columns, and metadata
- Indexing and slicing with `iloc` and `loc`
- Filtering data using conditions
- Sorting rows and indexes
- Adding, removing, and updating data
- Understanding data types
- Performing statistical analysis
- Handling missing values
- Working with unique and duplicate data
- Replacing, mapping, and encoding values

- Advanced techniques such as merging, joining, grouping, concatenation, and segmentation

Most importantly, we discovered why DataFrames are at the heart of modern data analysis. They provide a flexible and powerful way to organize structured data, making it easier to clean, explore, transform, visualize, and analyze information.

In the course on **Data Analytics, Data Science, and Machine Learning**, many of these techniques will reappear in much greater depth. The foundation built here will make those advanced topics significantly easier to understand and apply.

References

<https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.html>

<https://www.geeksforgeeks.org/pandas/python-pandas-dataframe/>

Next Section:

</> [32. Python: Matplotlib](#)